Лабораторна робота 3. ASP.NET Core Razor Pages

Теоретичні відомості та пояснення прикладів

Лабораторна робота розвиває застосунок каталогу фільмів. У ній додаються облікові записи користувачів, роль адміністратора, публічний каталог, завантаження постерів і відгуки. Головна ідея: публічні сторінки показують дані всім, а адміністративні сторінки змінюють дані лише після перевірки прав доступу.

Як частини застосунку пов'язані між собою

Уявімо сайт кінотеатру. Відвідувач відкриває каталог і бачить фільми. Після входу він може поставити оцінку. Адміністратор додає нові фільми та їхні постери. Для цього застосунок працює одночасно з браузером, базою даних і файлами на сервері.

Частина	За що відповідає	Приклад у роботі
Razor Page `.cshtml`	Формує HTML, який отримає браузер	Картки фільмів і форма відгуку
PageModel `.cshtml.cs`	Обробляє запит і готує дані сторінки	Пошук фільму за `id`
Модель у `Models`	Описує властивості даних	`Movie`, `Review`
Контекст у `Data`	Читає та зберігає дані через EF Core	`RazorPagesMovieContext`
Identity	Керує обліковими записами та входом	`UserManager`, `SignInManager`
Файлова система	Зберігає байти зображень	Постери у `wwwroot/uploads`

Модель `Movie` і клас `DetailsModel : PageModel` мають різне призначення. `Movie` описує один фільм. `DetailsModel` готує цілу сторінку: фільм, відгуки, середню оцінку, поля форми й обробники запитів.

Спрощена структура нових і змінених файлів:

```
RazorPagesMovie/
├── Program.cs
├── appsettings.json
├── Data/
│   ├── RazorPagesMovieContext.cs
│   ├── ApplicationDbContext.cs
│   └── IdentitySeeder.cs
├── Models/
│   ├── Movie.cs
│   └── Review.cs
├── Pages/
│   ├── _ViewImports.cshtml
```

```
│   ├── Movies/                  # керування каталогом: лише Admin
│   ├── Catalog/
│   │   ├── Index.cshtml
│   │   ├── Index.cshtml.cs
│   │   ├── Details.cshtml
│   │   └── Details.cshtml.cs
│   └── Shared/
│       ├── _Layout.cshtml
│       ├── _LoginPartial.cshtml
│       └── _ValidationScriptsPartial.cshtml
└── wwwroot/
    ├── images/placeholder-poster.svg
    └── uploads/movies/           # папки й постери створює застосунок
```

Папки міграцій додаються окремо для відповідних контекстів. Сторінки Identity можуть працювати з бібліотеки, тому відсутність локальної папки `Areas/Identity` сама по собі не є помилкою.

Мінімум синтаксису для читання коду

```
public string? Comment { get; set; }
```

`public` дозволяє звертатися до властивості з іншого коду. `string` означає текст, а `?` дозволяє значення `null`, тобто відсутність тексту. `get` дає змогу прочитати значення, `set` - записати його.

```
public IList<Movie> Movies { get; private set; } = new List<Movie>();
```

Тут `Movies` - колекція об'єктів `Movie`. Сторінка може її читати, але замінити колекцію може лише код самого класу, бо `set` має модифікатор `private`. `new List<Movie>()` створює порожній список.

У виразах `m => m.Title` ліва частина називає поточний елемент, а права визначає дію над ним. Наприклад, `OrderBy(m => m.Title)` означає «впорядкувати фільми за назвою». Це лямбда-вираз.

`async` і `await` використовуються для очікування операцій, наприклад запиту до БД. `await` не означає «виконати й забути»: наступний рядок цього методу виконається після завершення операції. Під час очікування вводу-виводу сервер може використовувати потік для інших запитів.

Оператор `?.` безпечно звертається до властивості, якщо об'єкт може бути `null`. Вираз `Movie?.Title ?? "Фільм"` означає «взяти назву, якщо вона є, інакше показати слово "Фільм"». Оператор `!` у `Movie!.Title` лише приглушує попередження компілятора: він не створює об'єкт і не захищає від помилки під час виконання.

1. Автентифікація та авторизація

Автентифікація відповідає на запитання: «Хто це?». Під час входу система перевіряє облікові дані. Реєстрація створює обліковий запис і залежно від налаштувань може завершуватися автоматичним входом. У cookie-схемі цієї лабораторної після успішного входу сервер видає захищену cookie, яку браузер надсилає з наступними запитами.

Авторизація відповідає на запитання: «Що цьому користувачеві дозволено?». Наприклад, користувач може переглядати каталог, а лише адміністратор може створювати, редагувати та видаляти фільми.

Роль - назва групи прав. У роботі використовується роль `Admin`.

```
Вхід користувача → система знає його особу → перевірка ролі → дозвіл або заборона дії
```

У конвеєрі ASP.NET Core порядок важливий:

```
app.UseAuthentication(); // визначає, хто виконав запит
app.UseAuthorization();  // перевіряє, чи має він право доступу
```

Чому не потрібно вводити пароль на кожній сторінці

HTTP-запити окремі: відкриття каталогу, деталей і надсилання відгуку є різними запитами. Браузер додає cookie автентифікації, а сервер перевіряє її та відновлює відомості про користувача. Пароль при кожному переході повторно не передається.

У C# поточного користувача представляє `User`. Перевірка `User.Identity?.IsAuthenticated == true` показує, чи запит належить користувачеві, який увійшов. `User.IsInRole("Admin")` перевіряє роль.

Спроба відкрити захищену сторінку дає різний результат: гостя зазвичай перенаправляють на вхід; користувач, який уже увійшов, але не має потрібної ролі, отримує відмову в доступі. Реєстрація сама по собі не надає роль `Admin`.

2. ASP.NET Core Identity

ASP.NET Core Identity - готова підсистема для облікових записів: реєстрації, входу, виходу, хешування паролів, ролей і токенів.

У лабораторній додаються пакети:

```
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
dotnet add package Microsoft.AspNetCore.Identity.UI
```

- `Identity.EntityFrameworkCore` зберігає користувачів, ролі та зв'язки між ними через Entity Framework Core.
- `Identity.UI` надає готові Razor Pages для `/Identity/Account/Login`, `/Register` і `/Logout`.

Зазвичай ці сторінки не потрібно створювати вручну. Посилання на них можна розмістити у `_LoginPartial.cshtml`.

Що роблять основні класи Identity

`IdentityUser` описує обліковий запис: ідентифікатор, ім'я, email та службові поля. Identity зберігає хеш пароля. Хешування дає змогу перевірити введений пароль без зберігання його відкритим текстом.

`UserManager<IdentityUser>` працює з обліковими записами: створює користувача, шукає його, змінює пароль, додає роль. `SignInManager<IdentityUser>` керує входом і виходом. `RoleManager<IdentityRole>` керує ролями. Це різні обов'язки, тому в коді є кілька служб.

```
builder.Services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders()
    .AddDefaultUI();
```

Цей ланцюжок реєструє Identity з користувачами й ролями, визначає контекст для зберігання, додає провайдери токенів для таких операцій, як відновлення пароля, і підключає готовий інтерфейс. Сам по собі він не створює таблиці: для цього потрібні міграції.

Для прикладів слід використовувати сумісні з цільовою версією .NET пакети. Номер версії зі знімка екрана лабораторної не обов'язково відповідає вашому проєкту.

3. Два DbContext в одному застосунку

У роботі використано два контексти даних:

```
RazorPagesMovieContext  → фільми та відгуки
ApplicationDbContext    → користувачі, ролі, паролі, claims
```

DbContext - клас Entity Framework Core, який представляє сеанс роботи з базою даних. Через його властивості `DbSet<T>` застосунок читає та змінює записи.

`ApplicationDbContext` успадковується від `IdentityDbContext<IdentityUser>`. Отже, він уже містить схему таблиць Identity. Обидва контексти можуть використовувати одну фізичну базу даних і один рядок підключення, але для кожного контексту створюються окремі міграції.

Клас, об'єкт і таблиця

Клас `Movie` описує поля фільму. Об'єкт цього класу в пам'яті містить конкретний фільм. Рядок таблиці зберігає відповідні значення у БД. EF Core виконує перетворення між об'єктами C# і даними таблиць; це називають об'єктно-реляційним відображенням, або ORM.

`_context.Movie.Add(movie)` позначає об'єкт як новий. `await _context.SaveChangesAsync()` надсилає зміни до БД. Просте присвоєння властивості ще не означає, що база вже оновилася.

Рядок підключення визначає, де база розташована. Для SQLite це зазвичай шлях до файлу; для SQL Server він містить адресу сервера, назву бази та параметри підключення. У цій лабораторній

використовується реляційна БД із міграціями. Вона відрізняється від демонстраційного провайдера InMemory з лекції.

Як контекст потрапляє в PageModel

```
private readonly RazorPagesMovieContext _context;

public IndexModel(RazorPagesMovieContext context)
{
    _context = context;
}
```

ASP.NET Core бачить параметр конструктора, отримує зареєстрований контекст зі служб і передає його в `IndexModel`. Це впровадження залежностей, або DI. Поле `_context` зберігає посилання на отриманий об'єкт. Модифікатор `readonly` забороняє перевизначити поле поза ініціалізацією чи конструктором, але не забороняє працювати з базою через контекст.

Зазвичай `AddDbContext` надає контекст на час одного HTTP-запиту. Його не слід сприймати як спільну глобальну змінну для всіх користувачів.

4. Роль Admin і захист папки сторінок

Політика авторизації формулює правило доступу:

```
options.AddPolicy("RequireAdministratorRole", policy =>
    policy.RequireRole("Admin"));
```

Потім правило можна застосувати до всієї папки Razor Pages:

```
options.Conventions.AuthorizeFolder("/Movies", "RequireAdministratorRole");
```

Це захищає всі сторінки в `Pages/Movies`: список, створення, редагування та видалення. Не потрібно дублювати `[Authorize]` у кожному класі.

`[AllowAnonymous]` застосовують там, де сторінка має бути доступною без входу, наприклад у `Pages/Catalog/Index.cshtml.cs` і `Details.cshtml.cs`.

> Приховати посилання «Movies (Admin)» у меню недостатньо. Це лише змінює інтерфейс. Справжній захист створюють політика, атрибут `[Authorize]` або конвенція `AuthorizeFolder`.

`RequireAdministratorRole` - довільне ім'я політики, яке має однаково писатися під час реєстрації та використання. `Admin` - назва ролі, яку перевіряє ця політика. Шлях `/Movies` у конвенції означає папку сторінок `Pages/Movies`, а не таблицю БД.

Сторінка `Pages/Catalog/Details.cshtml` може читати ту саму таблицю фільмів, що й адміністративні сторінки. Поділ на `Movies` і `Catalog` організовує інтерфейс і права доступу, а не створює дві копії каталогу в БД.

Особливість Razor Pages: `[Authorize]` застосовують до класу `PageModel` або через конвенції. Атрибут над окремим `OnPost...` не забезпечує авторизацію цього обробника. Якщо сторінка публічна, а її POST-дії потребують входу, перевірку треба реалізувати окремо. Детальний приклад наведено в розділі 13. Документація Microsoft

5. Початковий адміністратор і сідер

Сідер - код, який створює початкові дані під час запуску застосунку. У цій роботі `IdentitySeeder` перевіряє існування ролі `Admin`, створює її за потреби, створює адміністратора та додає його до ролі.

Для цього використовуються служби Identity:

- `RoleManager<IdentityRole>` - створює й перевіряє ролі;
- `UserManager<IdentityUser>` - створює користувачів, шукає їх і додає до ролей.

Сідер викликають до `app.Run()`, усередині області служб `CreateScope()`. Пароль із `appsettings.json` допустимий лише для навчального прикладу. У реальному застосунку секрети не зберігають у файлі, який потрапляє до репозиторію.

Чому сідер перевіряє існування записів

Застосунок запускається багато разів. Якби кожен запуск без перевірки створював адміністратора, база містила б дублікати або запуск завершувався б помилкою. Сідер має бути ідемпотентним: повторний виклик не повинен повторно створювати вже наявні записи.

Роль, користувач і належність користувача до ролі - окремі записи. Створений користувач ще не є адміністратором, доки його не додано до `Admin`.

`CreateScope()` створює тимчасову область служб, у якій сідер може користуватися контекстом БД і менеджерами Identity. `using` звільняє ресурси після завершення. Таблиці Identity мають існувати до запуску сідера: спочатку застосовують міграцію, потім запускають застосунок.

Методи `CreateAsync` і `AddToRoleAsync` повертають результат операції. Потрібно перевіряти `Succeeded` та `Errors`: наприклад, пароль може не відповідати вимогам. У сідері з лабораторної зміна пароля в конфігурації не змінить пароль уже створеного адміністратора, бо створення відбувається лише за його відсутності.

Для локальних паролів можна використати .NET User Secrets. Це зберігання налаштувань поза файлом проєкту, а не спосіб шифрування секретів для робочого сервера.

6. Міграції Entity Framework Core

Міграція - опис змін структури бази даних: створення таблиці, додавання колонки, індексу або зовнішнього ключа. Команда створення міграції не змінює базу відразу, а записує зміни в C#-файли. Команда `database update` застосовує їх до бази.

```
dotnet ef migrations add CreateIdentitySchema -c ApplicationDbContext -o
Data/Migrations/Identity
dotnet ef database update -c ApplicationDbContext
```

Параметр `-c` указує конкретний DbContext. Це необхідно, бо їх два. Після зміни моделі `Movie` або додавання `Review` потрібно створити нову міграцію для `RazorPagesMovieContext` і знову виконати `database update`.

Приклад: додавання шляху до постера

1. До `Movie` додається `public string? PosterPath { get; set; }`.
2. Команда `migrations add MoviePoster -c RazorPagesMovieContext` порівнює поточну модель зі знімком попередньої моделі й створює опис зміни.
3. Після перегляду міграції команда `database update -c RazorPagesMovieContext` додає колонку до БД.

Міграція зазвичай має `Up()` для застосування змін та `Down()` для їх скасування. Знімок моделі, або snapshot, потрібен EF Core для наступних порівнянь. Історія застосованих міграцій дозволяє не виконувати їх повторно.

Параметр `-o` визначає папку C#-файлів міграцій, а не розташування БД. Для кожного контексту варто послідовно використовувати одну вибрану папку. Наявну початкову міграцію фільмів із лабораторної 2 повторно створювати не потрібно.

SQL Server і SQLite використовують різні провайдери. Заміна `UseSqlServer` на `UseSqlite` не гарантує сумісності вже створених міграцій. Для роботи оберіть один провайдер і використовуйте його послідовно. Міграції для різних провайдерів

Команди `dotnet ef` виконуються з папки потрібного проєкту, де є `.csproj`. Інструмент `dotnet-ef` і пакет `Microsoft.EntityFrameworkCore.Design` мають бути доступні та сумісні з використаною версією EF Core. Якщо таблиць немає, спершу перевірте вибраний контекст і рядок підключення, а не видаляйте базу.

7. Публічний каталог і запити EF Core

Папка `Pages/Catalog` містить публічні сторінки:

```
Catalog/Index      – перелік фільмів, пошук і фільтр
Catalog/Details    – деталі одного фільму, постер і відгуки
```

`AsNoTracking()` використовують для даних, які тільки відображаються. EF Core не відстежує такі об'єкти, тому запит виконується економніше.

```
var query = _context.Movie.AsNoTracking().AsQueryable();
```

`IQueryable` ще не містить даних у пам'яті. Він описує майбутній SQL-запит. До нього можна поступово додати умови `Where`, сортування `OrderBy`, а реальне виконання відбувається під час `ToListAsync()`.

Як читається запит

```
var query = _context.Movie.AsNoTracking();

if (!string.IsNullOrWhiteSpace(genre))
{
    query = query.Where(m => m.Genre == genre);
}

Movies = await query.OrderBy(m => m.Title).ToListAsync();
```

Спочатку описано читання фільмів. Якщо жанр вибраний, додається умова відбору. Потім додається сортування, запит виконується, а результат стає списком `Movies`. Якщо викликати `ToListAsync()` надто рано, подальше фільтрування списку виконуватиметься вже в пам'яті застосунку.

`AsNoTracking()` доречний для показу каталогу. Для редагування зручно завантажити сутність із відстеженням: EF Core помітить змінені властивості й збереже їх. `AsNoTracking()` не є обмеженням прав і не забороняє іншим частинам застосунку змінювати ці записи.

Сторінка одного фільму

```
@page "{id:int}"
```

Ця директива дозволяє адресу `/Catalog/Details/12`, де `12` передається як `id`. Обмеження `int` вимагає ціле число, але не гарантує, що фільм із таким номером існує.

```
var movie = await _context.Movie.AsNoTracking()
    .FirstOrDefaultAsync(m => m.Id == id);

if (movie is null)
{
    return NotFound();
}
```

`FirstOrDefaultAsync` повертає перший відповідний запис або `null`. `NotFound()` формує відповідь 404. Завдяки перевірці код не намагається показати властивості неіснуючого фільму.

8. Пошук і фільтрація через GET

Для каталогу пошук і жанр передаються в URL, наприклад:

```
/Catalog?q=Matrix&genre=Action
```

Властивість із `[BindProperty(SupportsGet = true)]` дозволяє Razor Pages заповнити її значенням із рядка запиту:

```
[BindProperty(SupportsGet = true)]
public string? q { get; set; }
```

GET доречний для пошуку, бо він не змінює дані. Таке посилання можна додати в закладки або надіслати іншій людині.

`SelectList` містить варіанти для HTML-елемента `<select>`. Жанри доцільно отримати з бази, прибрати порожні значення, залишити унікальні через `Distinct()` і відсортувати.

Що саме надсилає форма

У `<input name="q">` атрибут `name` задає назву параметра запиту. У формі з `method="get"` браузер додає такі параметри після `?` в URL. Символ `&` розділяє параметри. `SupportsGet = true` дозволяє заповнювати властивість PageModel під час GET; звичайний `[BindProperty]` за замовчуванням цього не робить.

`OnGetAsync()` щоразу читає актуальні параметри та будує запит заново. Порожній жанр означає, що умову за жанром не додають. Якщо задані і пошук, і жанр, фільм має відповідати обом умовам.

У `EF.Functions.Like(m.Title, $"%{q}%")` знак `%` означає довільну послідовність символів: шаблон `%Matrix%` шукає назви з таким фрагментом. Регістр і правила зіставлення залежать від БД та її налаштувань. LINQ-параметри передає провайдер EF Core; не потрібно вручну складати SQL-рядок із введеного тексту.

`asp-items="Model.GenreOptions"` формує варіанти списку, а `asp-for="genre"` пов'язує вибране значення з властивістю. Це приклад Tag Helpers: сервер обробляє `asp-*`, і браузер отримує звичайний HTML.

9. Спільний макет і часткові подання

`Pages/Shared/_Layout.cshtml` - **спільний макет** усіх сторінок. Він містить HTML-каркас, навігацію, підключення Bootstrap, CSS і JavaScript. Конструкція `@RenderBody()` вставляє в цей каркас вміст конкретної Razor Page.

`_LoginPartial.cshtml` - **часткове подання**. Воно містить невеликий фрагмент інтерфейсу: посилання «Увійти» і «Реєстрація» для гостя або привітання та «Вийти» для автентифікованого користувача. Макет вставляє його так:

```
<partial name="_LoginPartial" />
```

За допомогою `User.IsInRole("Admin")` макет може показувати адміністративне посилання лише адміністраторові.

`_ViewStart.cshtml` зазвичай задає `Layout = "_Layout"`. `_ViewImports.cshtml` підключає спільні простори імен і Tag Helpers. Часткове подання не має власної адреси: його вставляють у сторінку або макет.

```
@inject SignInManager<IdentityUser> SignInManager
```

Директива `@inject` отримує службу з DI безпосередньо в Razor-файлі. Наприклад, `SignInManager.IsSignedIn(User)` дозволяє вибрати, які елементи меню показати.

Вихід у лабораторній виконують POST-формою. Form Tag Helper додає антифальсифікаційний токен, який сервер перевіряє під час POST. Він захищає від певних підроблених запитів з іншого сайту, але не замінює перевірку ролі чи власника запису.

`_ValidationScriptsPartial` підключає скрипти перевірки форми. Їхня задача - швидко показати помилки користувачеві; остаточно дані перевіряє сервер.

10. Завантаження постера

Для надсилання файла звичайною HTML-формою використовують POST і атрибут:

```
enctype="multipart/form-data"
```

У `PageModel` файл приймають через `IFormFile`:

```
[BindProperty]
public IFormFile? PosterFile { get; set; }
```

`IWebHostEnvironment.WebRootPath` дає фізичний шлях до папки `wwwroot`. Зображення зберігають у `wwwroot/uploads/movies/<id>`, а в базі даних тримають лише веб-шлях, наприклад:

```
/uploads/movies/12/poster-a1b2c3.webp
```

`wwwroot` призначена для файлів, доступних браузеру. Тому `PosterPath` можна поставити у `src` зображення на публічній сторінці.

Під час завантаження потрібно перевірити:

- розширення або фактичний тип файлу;
- розмір файлу;
- успішність збереження;
- унікальність назви файлу.

`Guid.NewGuid()` створює практично унікальне ім’я й зменшує ризик перезаписати чужий файл. Не слід використовувати початкове ім’я файлу як ім’я на сервері.

Файл у запиті ще не є збереженим постером

`IFormFile` представляє файл, який надійшов із запитом. `Length` дає розмір у байтах, а `CopyToAsync(stream)` копіює вміст у вибраний потік. Застосунок сам вирішує, чи прийняти файл і куди записати його.

```
Вибір файла у браузері
    → POST із multipart/form-data
    → IFormFile у PageModel
    → серверна перевірка
    → запис файла на диск
    → запис PosterPath у БД
```

Фізичний шлях на кшталт `.../wwwroot/uploads/movies/12/poster.webp` потрібен серверу для запису файла. Веб-шлях `/uploads/movies/12/poster.webp` потрібен браузеру для нового GET-запиту за зображенням. У базі зберігають саме веб-шлях, тому вона не містить байтів постера.

Перевірки та показ зображення

`accept=".jpg,.jpeg,.png,.webp"` допомагає користувачеві вибрати файл, але не доводить, що файл безпечний або справді є зображенням. Сервер має перевірити розмір, дозволене розширення та вміст; заголовку `ContentType` теж не можна безумовно довіряти. У прикладі PDF перевірка розширення й розміру є лише початковим рівнем перевірки. Завантаження файлів в ASP.NET Core

Постери в цій роботі публічні. Захист `/Movies` обмежує операції адміністратора, але не робить файл у `wwwroot/uploads` приватним. Для завантажених під час роботи файлів потрібне налаштоване обслуговування статичних файлів, як `UseStaticFiles()` у лабораторній; при переході на нові шаблони не слід припускати, що `MapStaticAssets()` автоматично охоплює нові завантаження.

Якщо `PosterPath` порожній, сторінка показує плейсхолдер. Проста перевірка порожнього рядка не виявить ситуацію, коли шлях є, але сам файл уже видалено.

Чому операції з БД і файлами треба узгоджувати

У прикладі створення спочатку зберігають фільм, щоб отримати його `Id`, потім записують постер у папку цього фільму й зберігають шлях. Якщо завантаження не вдалося, запис фільму вже може існувати. Повторне надсилання форми здатне створити дубль.

Заміна постера має аналогічний ризик: видалення старого файла до успішного збереження нового шляху може залишити БД із посиланням на відсутнє зображення. Доцільно спочатку зберегти новий файл і шлях, а потім прибрати старий. Потрібно також обробляти невдалі операції й очищати зайві файли. Транзакція БД сама по собі не скасовує записів у файловій системі.

11. Редагування та захист від overposting

Overposting виникає, коли клієнт надсилає властивості, які форма нібито не повинна змінювати, але сервер автоматично прив’язує їх до сутності.

Безпечніший підхід під час редагування:

1. Знайти наявний об'єкт у базі за `Id`.
2. Оновити лише дозволені властивості, наприклад `Title`, `Genre`, `Price`.
3. Зберегти зміни.

```
var dbMovie = await _context.Movie.FirstOrDefaultAsync(m => m.Id == Movie.Id);
if (dbMovie is null) return NotFound();

dbMovie.Title = Movie.Title;
dbMovie.Genre = Movie.Genre;
await _context.SaveChangesAsync();
```

Такий підхід не дає клієнту непомітно змінити службові поля, яких немає у формі.

Наприклад, браузер може надіслати довільний `PosterPath`, хоча форма містить лише вибір файла. Сервер має формувати цей шлях після збереження перевіреного зображення. Прихований `<input type="hidden">` також можна змінити: він не є захищеним сховищем.

Окрема модель вводу допомагає явно обмежити поля, які приймає форма. Незалежно від прив'язування, сервер перевіряє права на об'єкт і допустимість значень. Перевірка `ModelState.IsValid` відповідає за дані, а перевірка ролі чи власника - за право їх змінювати.

12. Відгук, оцінка і зв'язок між таблицями

Модель `Review` описує один відгук:

```
Review
├── MovieId       – до якого фільму належить відгук
├── UserId        – хто його створив
├── Rating        – оцінка від 1 до 5
├── Comment       – текст відгуку
├── CreatedAt     – час створення
└── UpdatedAt     – час останнього редагування
```

`MovieId` є зовнішнім ключем до фільму. Колекція `Movie.Reviews` дозволяє отримати всі відгуки певного фільму.

Унікальний індекс для пари `MovieId` і `UserId` гарантує правило: один користувач може залишити лише один відгук до одного фільму.

```
modelBuilder.Entity<Review>()
    .HasIndex(r => new { r.MovieId, r.UserId })
    .IsUnique();
```

`DeleteBehavior.Cascade` означає: коли фільм видаляють, база автоматично видаляє і його відгуки.

Зв'язок «один до багатьох»

Один фільм може мати багато відгуків. Кожен відгук має один `MovieId`, який указує на фільм. Наприклад, для фільму з `Id = 12` відгуки користувачів А і Б міститимуть однаковий `MovieId = 12`, але різні `UserId`.

MovieId	UserId	Rating	Чи можна додати запис
12	user-A	5	Так, якщо такого відгуку ще немає
12	user-B	4	Так, це інший користувач
12	user-A	3	Ні як новий запис; потрібно оновити відгук А
15	user-A	3	Так, це інший фільм

Унікальний індекс працює після застосування відповідної міграції. Перевірка існування відгуку в C# дає потрібну поведінку інтерфейсу, а обмеження БД захищає від дублів, зокрема при одночасних запитах.

`Review.Movie` і `Movie.Reviews` - навігаційні властивості для переходу між пов'язаними об'єктами. Саме оголошення колекції не завантажує відгуки автоматично. У лабораторній це робить `Include(m => m.Reviews)` разом із запитом фільму.

`Review.UserId` зберігає ідентифікатор Identity, але в наведеній конфігурації не створює автоматично зовнішній ключ до таблиці користувачів іншого контексту. Видалення користувача не означатиме автоматичного видалення його відгуків. Каскадне видалення в прикладі налаштовано саме між фільмом і відгуками.

`ReviewerName` зберігає копію відображуваного імені на момент запису. Зміна імені облікового запису не обов'язково змінить старі відгуки. `CreatedAt` і `UpdatedAt` зручно записувати в UTC. Виклик `ToLocalTime()` у серверному Razor-коді використовує часовий пояс сервера, який може відрізнятися від поясу користувача.

Текстове поле `Movie.Rating` із попередніх робіт і числова оцінка `Review.Rating` - різні дані. Середню користувацьку оцінку обчислюють саме з відгуків.

13. Власник відгуку та адміністратор

Для створення або редагування відгуку сервер має перевірити, що користувач увійшов. `UserId` беруть із поточного облікового запису. Значення з прихованого поля форми не можна використовувати як доказ авторства.

Публічний GET і захищений POST на одній сторінці

У лабораторній `DetailsModel` має `[AllowAnonymous]`, щоб усі бачили фільм. Водночас над методами збереження й видалення стоять `[Authorize]`. Для Razor Pages така комбінація не забезпечує очікуваного захисту: атрибут над окремим handler не застосовується як правило авторизації. Попередня коротка версія цього конспекту також неточно описувала цей момент.

Один із варіантів архітектури - винести дії з відгуками на окрему захищену сторінку. Якщо зберігати форму на публічній `Details`, на початку кожного POST-обробника потрібно явно перевірити користувача та потрібні права. Для складніших правил можна залучити службу авторизації. Авторизація Razor Pages

Нижче наведено фрагменти для наявного `DetailsModel`, який отримує `_userManager` і `_context` через конструктор. Вони пояснюють перевірки, а не є окремим готовим класом.

```
// На початку OnPostSaveReviewAsync і OnPostDeleteReviewAsync:
if (User.Identity?.IsAuthenticated != true)
{
    return Challenge();
}

var user = await _userManager.GetUserAsync(User);
if (user is null)
{
    return Challenge();
}

var userId = user.Id;
```

`Challenge()` просить налаштовану схему автентифікації розпочати вхід. Для cookie-схеми вебсторінки це зазвичай перенаправлення на форму входу. Перевірка `user is null` потрібна, бо відповідного облікового запису вже може не бути в БД.

Для видалення відгуку слід перевірити одну з двох умов:

```
поточний користувач є автором відгуку
або
поточний користувач має роль Admin
```

Якщо обидві умови хибні, сервер повертає `Forbid()`. Така перевірка має бути у C#-обробнику, навіть якщо кнопка «Видалити» не відображається чужим користувачам.

```
// Після перевірки користувача в OnPostDeleteReviewAsync:
var review = await _context.Reviews
    .FirstOrDefaultAsync(r => r.Id == reviewId);

if (review is null)
{
    return NotFound();
}
```

```
var isOwner = review.UserId == userId;
var isAdmin = User.IsInRole("Admin");

if (!isOwner && !isAdmin)
{
    return Forbid();
}

_context.Reviews.Remove(review);
await _context.SaveChangesAsync();

return RedirectToPage("./Details", new { id = review.MovieId });
```

`!isOwner && !isAdmin` читається як «не власник і не адміністратор». `Forbid()` означає відмову в доступі; у cookie-схемі користувач може потрапити на сторінку Access Denied. `NotFound()` означає відсутність запису. Це різні причини, тому для них різні результати.

Як форма обирає обробник

```
<form method="post" asp-page-handler="SaveReview">
    <input type="hidden" name="id" value="@Model.id" />
    <textarea name="MyComment">@Model.MyComment</textarea>
    <button type="submit">Зберегти відгук</button>
</form>
```

Це скорочений фрагмент; поле оцінки додається окремо. Значення `SaveReview` указує Razor Pages вибрати метод `OnPostSaveReviewAsync`. Для `asp-page-handler="DeleteReview"` відповідним методом є `OnPostDeleteReviewAsync`. Браузер надсилає HTTP-запит, а назву C#-методу за цими правилами обирає сервер.

Атрибут `name="MyComment"` пов’язує надісланий текст із властивістю `[BindProperty] public string? MyComment { get; set; }`. Прихований `id` указує, до якого фільму належить дія, але сервер однаково перевіряє існування фільму та права користувача.

Перевірка оцінки та коментаря

Навіть якщо інтерфейс пропонує лише кнопки від 1 до 5, довільний клієнт може надіслати інше число. Тому правила задають на тих властивостях, до яких реально прив’язується форма:

```
[BindProperty]
[Required(ErrorMessage = "Оберіть оцінку")]
[Range(1, 5, ErrorMessage = "Оцінка повинна бути від 1 до 5")]
public int? MyRating { get; set; }

[BindProperty]
[StringLength(2000, ErrorMessage = "Коментар має містити не більше 2000 символів")]
public string? MyComment { get; set; }
```

Для цих атрибутів потрібні `Microsoft.AspNetCore.Mvc` і `System.ComponentModel.DataAnnotations`. Перед зміною БД перевіряють `ModelState.IsValid`.

Атрибут `[StringLength(2000)]` лише на `Review.Comment` не замінює перевірку `MyComment`. У цьому прикладі форма заповнює `MyComment`, а `Review` створюється вручну пізніше. EF Core не виконує автоматично всі правила Data Annotations над кожним об'єктом під час `SaveChangesAsync()`.

Якщо дані неправильні, перед `return Page()` потрібно знову завантажити фільм і список відгуків для відображення. `Page()` не викликає `OnGetAsync()` автоматично. Зручно винести завантаження даних у допоміжний метод і викликати його з GET та з гілки помилок POST. Він не повинен замінювати введені `MyRating` і `MyComment` старими значеннями з БД.

Під час показу `@r.Comment` Razor за замовчуванням кодує HTML-символи. Це дозволяє показати введений текст без виконання його як HTML-розмітки. Не використовуйте `Html.Raw` для неперевірених коментарів.

Повний шлях одного відгуку

1. Користувач відкриває `/Catalog/Details/12`. GET завантажує фільм і відгуки.
2. Користувач обирає оцінку та надсилає форму. POST містить поля форми, cookie й антифальсифікаційний токен.
3. Сервер прив'язує поля до властивостей і викликає `OnPostSaveReviewAsync`.
4. Обробник перевіряє вхід, дані форми та існування фільму.
5. Пошук за парою `MovieId` і `UserId` визначає, створити відгук чи оновити наявний.
6. `SaveChangesAsync()` зберігає зміни.
7. `RedirectToPage` повертає браузер на GET тієї самої сторінки, де вже видно нові дані.

Цей порядок називають Post/Redirect/Get. Оновлення сторінки після перенаправлення повторює читання, а не надсилання форми.

Середню оцінку обчислюють із колекції відгуків:

```
var average = reviews.Count > 0 ? reviews.Average(r => r.Rating) : 0;
```

Для оцінок 5, 4 і 3 результат дорівнює 4. Умова перед `Average` обробляє порожній список. Якщо відгуків немає, краще показати «Ще немає оцінок», щоб умовний нуль не сприймався як реальна оцінка.

Обчислену властивість також потрібно вставити в Razor-розмітку. Саме присвоєння `AverageRating` у PageModel ще не показує її користувачеві:

```
@if (Model.ReviewsCount > 0)
{
    <p>Середня оцінка: @Model.AverageRating.ToString("0.0") / 5</p>
    <p>Кількість відгуків: @Model.ReviewsCount</p>
}
```

14. Що перевірити перед здаванням

1. Анонімний відвідувач відкриває `/Catalog` і `/Catalog/Details/{id}`.
2. Анонімний відвідувач не може відкрити `/Movies`.
3. Адміністратор входить, створює, редагує та видаляє фільми.
4. Пошук і фільтр жанру працюють через URL-параметри.
5. Постер відображається у каталозі, а за його відсутності показується плейсхолдер.
6. Завантаження відхиляє неприпустимий або надто великий файл.
7. Користувач створює лише один власний відгук до одного фільму, може його оновити або видалити.
8. Інший користувач не може видалити чужий відгук, а адміністратор може.

Як перевіряти, щоб розуміти результат

Для доступу зручно мати три стани: гість, звичайний користувач і Admin. Для перевірки власності потрібні два різні звичайні користувачі. Зміна стану входу в одному вікні може вплинути на всі вкладки цього браузера, тому для незалежного сеансу можна використати приватне вікно.

Перевіряйте і посилання, і пряме введення URL. Якщо посилання приховане, але прямий перехід на адміністративну сторінку працює для гостя, авторизація не налаштована належним чином.

Для форм перевірте порожні поля, завелику оцінку, довгий коментар і повторне збереження відгуку. Для постерів - відсутність файла, коректне зображення, перевищення розміру й заміну наявного постера. Серверна перевірка має працювати незалежно від JavaScript у браузері.

Типові помилки та їхній зміст

Симптом	Що перевірити
EF повідомляє про кілька контекстів	Чи задано `-c ApplicationDbContext` або `-c RazorPagesMovieContext`
Немає таблиці користувачів чи колонки PosterPath	Чи застосовано міграцію до тієї БД, яку використовує застосунок
Сідер не може створити адміністратора	Чи існують таблиці, чи відповідає пароль вимогам, що містить `IdentityResult.Errors`
Вхід працює, але `/Movies` недоступна	Чи має обліковий запис роль Admin, чи однаково названо політику
`PosterFile` завжди `null`	Чи є `multipart/form-data`, поле файла з правильним ім'ям і `[BindProperty]`
Шлях постера є, зображення не видно	Чи існує файл і чи віддає сервер цей URL
Після неправильної оцінки зникають деталі фільму	Чи завантажені дані сторінки перед `return Page()`
Анонімний POST виконує дію	Чи є реальна серверна перевірка, а не лише `[Authorize]` над handler
Браузер отримує 400 на POST	Зокрема перевірити антифальсифікаційний токен і cookie; не вимикати захист для приховування помилки

15. Питання для перевірки розуміння

1. Чому вхід і дозвіл редагувати фільм є різними перевірками?
2. Чому двом DbContext не обов'язково потрібні дві фізичні бази?
3. Яка різниця між `migrations add` і `database update`?
4. Чому для пошуку використовують GET, а для збереження відгуку - POST?
5. На якому рядку LINQ-запит перетворюється на завантажений список?
6. Що збережеться в БД після завантаження постера: байти чи шлях?
7. Чому приховане поле `UserId` не доводить, хто створив відгук?
8. Як дозволити публічний перегляд фільму та перевірити доступ до його POST-дій?
9. Чому атрибута валідації лише на сутності Review може бути недостатньо?
10. Що станеться, якщо файл записано, а збереження його шляху в БД завершилося помилкою?

Короткий словник

Термін	Значення
Identity	підсистема користувачів, паролів, входу та ролей
Authentication	встановлення особи користувача
Authorization	перевірка права на дію або ресурс
Role	група прав, наприклад `Admin`
DbContext	клас EF Core для доступу до даних
Migration	контрольована зміна структури бази даних
`IFormFile`	файл, отриманий із HTML-форми
`wwwroot`	папка статичних файлів, доступних браузеру
`IQueryable`	запит до БД, який ще можна доповнювати
`AsNoTracking()`	читання даних без відстеження змін
Partial	повторно використовуваний фрагмент розмітки

Додатковий термін	Значення
Cookie автентифікації	Захищені дані, які браузер надсилає для підтвердження сеансу входу
Claims	Твердження про користувача, наприклад його ідентифікатор
DI	Передавання зареєстрованих залежностей класам, які їх потребують
Seeder	Код створення початкових даних
Зовнішній ключ	Поле, що посилається на пов’язаний запис
Унікальний індекс	Обмеження, яке забороняє повторення значення або комбінації значень
Handler	Метод PageModel, що обробляє певний запит
Model binding	Заповнення параметрів і властивостей даними HTTP-запиту
ModelState	Результати прив’язування даних та їх валідації
CSRF-токен	Частина захисту від підроблених запитів з іншого сайту

Матеріали для уточнення

- Вступ до ASP.NET Core Identity
- Авторизація та обмеження для обробників Razor Pages
- Міграції EF Core
- Валідація моделей ASP.NET Core
- Відстеження змін і AsNoTracking
- Безпечне завантаження файлів